FAHAD BIN SULTAN UNIVERSITY (FBSU) — TABUK, KSA

Smart Campus & ALPR Smart Parking Platform

Engineering Master Backlog, Technical Specifications & Delivery Telemetry

53	10	1	42	19%
TOTAL PLANNED TASKS	COMPLETED (DONE) ✅	IN PROGRESS ⏳	BACKLOG REMAINING 📋	OVERALL DELIVERY

19% COMPLETED

ACTIVE SPRINT FOCUS: P2-04 — User Profile & Vehicle Registration Management API (`feature/P2-04-profile-vehicles-api`) In Progress ⏳

PHASE-BY-PHASE EXECUTION PROGRESS

Phase	Phase Domain Description	Total	Done ✅	Active ⏳	Backlog 📋	% Done
Phase P0	Discovery & Strategic Planning	**1**	1	0	0	**100%**
Phase P1	Foundations & DevOps Infrastructure	**6**	6	0	0	**100%**
Phase P2	Identity, Security & Campus Core	**5**	3	1	1	**60%**
Phase P3	Frontend Architecture (React 19 + TypeScript)	**5**	0	0	5	**0%**
Phase P4	Financial Ledger & Tuition Offset Engine	**5**	0	0	5	**0%**
Phase P5	Smart Parking & ALPR Computer Vision Gate	**9**	0	0	9	**0%**
Phase P6	Academic Campus Services & Resource Pods	**7**	0	0	7	**0%**
Phase P7	Notifications, Governance & Telemetry Analytics	**6**	0	0	6	**0%**
Phase P8	Security Hardening, Compliance & Quality Assurance	**5**	0	0	5	**0%**
Phase P9	Production Deployment, CI/CD & Operations	**4**	0	0	4	**0%**
TOTAL PROJECT BACKLOG		**53**	**10**	**1**	**42**	**19%**

MASTER TASK INVENTORY & STATUS MATRIX

ID	Task Title & Scope	Priority	Cx	Status
`P0-01`	Project Discovery, Architecture Blueprint & Technical Planning	Critical	L	Done ✅

ID	Task Title & Scope	Priority	Cx	Status
P1-01	Monorepo Architecture Restructure & Design Showcase Preservation	High	M	Done ✅
P1-02	Frontend Application Scaffold (React 19 + TypeScript Strict)	High	M	Done ✅
P1-03	Backend Core Modular Monolith Skeleton (Spring Boot 3.3, Java 21 LTS)	High	M	Done ✅
P1-04	PostgreSQL Persistence, Flyway Migrations & Testcontainers Harness	High	M	Done ✅
P1-05	Containerization, Docker Compose Orchestration & Health Checks	High	M	Done ✅
P1-06	Continuous Integration Pipeline (GitHub Actions CI)	High	M	Done ✅
P2-01	Campus Core Schema (University Tenant, College, Building, Term)	High	M	Done ✅
P2-02	Identity, Security Roles & Vehicles Domain Schema	High	M	Done ✅
P2-03	Authentication API (Register, Login, Refresh, Logout) & RBAC	Critical	L	Done ✅
P2-04	User Profile & Vehicle Registration Management API	Medium	M	In Progress ⏳
P2-05	Development & Demonstration Seed Data Fixture	Low	S	Backlog 📋
P3-01	Design Tokens, Global CSS & Application Shell	High	L	Backlog 📋
P3-02	Internationalization (AR/EN), RTL/LTR Switching & Theme Toggle	High	M	Backlog 📋
P3-03	Typed API Client, Auth Context & Route Guards	High	M	Backlog 📋
P3-04	Authentication UI (Login & Registration Flow)	High	M	Backlog 📋

TASK SPECIFICATION: P0-01 PHASE P0 — DISCOVERY & STRATEGIC PLANNING

Project Discovery, Architecture Blueprint & Technical Planning

Git Branch: `docs/P0-01-project-discovery` · Target Pull Request: Merged into main (3df8350)

Core Objective & Technical Scope:

Exhaustive exploration and technical documentation of existing assets: Architectural audit, UI inventory, PRD, user journey flows, modular monolith architecture, database ERD, MVP scope, delivery roadmap, and risk register.

Priority Level	Critical	Complexity Rating	`L`
Execution Status	Done ✅	Target PR / Commit	`Merged into main (3df8350)`
Upstream Dependencies	None		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Perform detailed audit of existing codebase, legacy prototypes, and git history.
2. Establish baseline build stability and verify asset integrity.
3. Author architectural blueprints, decision records (ADRs), and database ERDs.
4. Formulate comprehensive delivery plan.md and master engineering backlog TASKS.md.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `plan.md`
- `TASKS.md`
- `docs/discovery/*`
- `docs/architecture/*`
- `docs/decisions/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- All technical specifications are consistent, complete, and peer-reviewed.
- Baseline project builds cleanly with zero missing dependencies.

VERIFICATION & TEST COVERAGE STRATEGY

- npm run build validation test passed

TASK SPECIFICATION: P1-01 PHASE P1 — FOUNDATIONS & DEVOPS INFRASTRUCTURE

Monorepo Architecture Restructure & Design Showcase Preservation

Git Branch: `chore/P1-01-repo-restructure` · Target Pull Request: PR (chore/P1-01-repo-restructure)

Core Objective & Technical Scope:
Restructure codebase into an enterprise monorepo layout (frontend/, backend/, design-system/, docs/, public/), preserve the 18-section Figma master showcase and 16:9 executive pitch deck, and eliminate asset duplication.

Priority Level	High	Complexity Rating	`M`
Execution Status	Done ✅	Target PR / Commit	`PR (chore/P1-01-repo-restructure)`
Upstream Dependencies	P0-01 Approved		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Execute git mv operations to cleanly migrate files while preserving complete commit history.
2. Resolve and update all relative asset paths across HTML, CSS, and JS files.
3. Clean obsolete development server hooks in root bundler configuration.
4. Establish global public assets directory and deduplicate 50MB of presentation slides.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `design-system/**`
- `frontend/`
- `backend/`
- `.gitignore`
- `README.md`
- `vite.config.js`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Zero file or commit history loss throughout repository migration.
- Design system showcase and presentation decks render with 100% visual fidelity.
- Clean root directory containing only root configuration and domain folders.

VERIFICATION & TEST COVERAGE STRATEGY

- Vite build validation
- Asset URL resolution checks
- Git status cleanliness audit

TASK SPECIFICATION: P1-02 PHASE P1 — FOUNDATIONS & DEVOPS INFRASTRUCTURE

Frontend Application Scaffold (React 19 + TypeScript Strict)

Git Branch: `feature/P1-02-frontend-scaffold` · Target Pull Request: PR (feature/P1-02-frontend-scaffold)

Core Objective & Technical Scope:
Initialize high-performance React 19 + TypeScript production application in frontend/ with strict typechecking, ESLint 9, Prettier, and Vitest testing suite.

Priority Level	High	Complexity Rating	`M`
Execution Status	Done ✅	Target PR / Commit	`PR (feature/P1-02-frontend-scaffold)`
Upstream Dependencies	P1-01, ADR-0002		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Bootstrap React 19 + TypeScript project using Vite 6.
2. Configure strict TypeScript compiler options and ESLint Flat Config.
3. Configure Vitest testing environment with React Testing Library and jsdom.
4. Establish enterprise folder architecture (app/, features/, shared/, styles/).
5. Implement standardized scripts: dev, build, lint, format, typecheck, test.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/package.json`
- `frontend/tsconfig.json`
- `frontend/vite.config.ts`
- `frontend/eslint.config.js`
- `frontend/src/**`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- All npm scripts (build, lint, typecheck, test) execute with zero warnings or errors.
- Unit test suite passes with jsdom environment configured.

VERIFICATION & TEST COVERAGE STRATEGY

- npm run lint
- npm run typecheck
- npm run test
- npm run build

TASK SPECIFICATION: P1-03 PHASE P1 — FOUNDATIONS & DEVOPS INFRASTRUCTURE

Backend Core Modular Monolith Skeleton (Spring Boot 3.3, Java 21 LTS)

Git Branch: `feature/P1-03-backend-skeleton` · Target Pull Request: PR (feature/P1-03-backend-skeleton)

Core Objective & Technical Scope:

Bootstrap Spring Boot 3.3 modular monolith with Java 21 LTS virtual threads, Actuator telemetry, SpringDoc OpenAPI/Swagger, RFC 7807 ProblemDetail exception handling, and ArchUnit architecture governance.

Priority Level	High	Complexity Rating	`M`
Execution Status	Done ✅	Target PR / Commit	`PR (feature/P1-03-backend-skeleton)`
Upstream Dependencies	P1-01, ADR-0001		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Initialize Maven multi-module structure with Java 21 and Spring Boot 3.3.4.
2. Implement GlobalExceptionHandler emitting RFC 7807 ProblemDetail responses.
3. Configure SpringDoc OpenAPI with Swagger UI at /swagger-ui/index.html.
4. Configure application profiles (dev, test, prod) with isolated configurations.
5. Define and enforce ArchUnit architecture rules preventing illegal cross-module dependencies.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/pom.xml`
- `backend/src/main/java/.../common/**`
- `backend/src/main/resources/application*.yml`
- `backend/src/test/**`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Backend starts successfully on port 8080 with Actuator health endpoint UP.
- Swagger UI loads at /swagger-ui/index.html.
- Validation failures return standard RFC 7807 problem responses with localized error messages.

VERIFICATION & TEST COVERAGE STRATEGY

- GlobalExceptionHandlerTest
- ArchUnitTest
- ContextLoadTest
- Maven verify

TASK SPECIFICATION: P1-04 PHASE P1 — FOUNDATIONS & DEVOPS INFRASTRUCTURE

PostgreSQL Persistence, Flyway Migrations & Testcontainers Harness

Git Branch: `feature/P1-04-database-flyway` · Target Pull Request: PR (feature/P1-04-database-flyway)

Core Objective & Technical Scope:
Establish PostgreSQL database connectivity, Flyway migration versioning (V1 baseline), and abstract Testcontainers integration test harness for real database testing.

Priority Level	High	Complexity Rating	`M`
Execution Status	Done ✅	Target PR / Commit	`PR (feature/P1-04-database-flyway)`
Upstream Dependencies	P1-03		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Add Spring Data JPA, PostgreSQL JDBC driver, and Flyway migration dependencies.
2. Configure HikariCP connection pool with environment variable overrides.
3. Author V1__baseline.sql migration enabling pgcrypto and btree_gist extensions.
4. Configure Hibernate with ddl-auto=validate to enforce Flyway authority.
5. Create AbstractIntegrationTest spinning up reusable Testcontainers PostgreSQL instance.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/pom.xml`
- `backend/src/main/resources/db/migration/V1__baseline.sql`
- `application.yml`
- `AbstractIntegrationTest.java`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Flyway migrations execute cleanly on clean database.
- Subsequent boots validate existing schema without schema drift.
- Integration tests run against real containerized PostgreSQL.

VERIFICATION & TEST COVERAGE STRATEGY

- Flyway baseline migration test
- Testcontainers repository integration tests

TASK SPECIFICATION: P1-05 PHASE P1 — FOUNDATIONS & DEVOPS INFRASTRUCTURE

Containerization, Docker Compose Orchestration & Health Checks

Git Branch: `feature/P1-05-docker-compose` · Target Pull Request: PR (feature/P1-05-docker-compose)

Core Objective & Technical Scope:
Deliver unified single-command full-stack containerization: Multi-stage Dockerfiles for backend and frontend (Nginx), PostgreSQL with persistent volumes, and health-check dependency ordering.

Priority Level	High	Complexity Rating	`M`
Execution Status	Done ✅	Target PR / Commit	`PR (feature/P1-05-docker-compose)`
Upstream Dependencies	P1-02, P1-03, P1-04		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Create multi-stage builder Dockerfile for Spring Boot with non-root runtime user.
2. Create multi-stage builder Dockerfile for React SPA with Nginx reverse proxy to /api.
3. Configure docker-compose.yml with postgres, backend, and frontend services on fbsu-network.
4. Configure container health checks with depends_on condition: service_healthy.
5. Author .env.example and comprehensive local development runbook.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/Dockerfile`
- `frontend/Dockerfile`
- `frontend/nginx.conf`
- `docker-compose.yml`
- `.env.example`
- `docs/runbooks/local-development.md`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- docker compose up boots all 3 containers to healthy state.
- Frontend serves UI on port 80; Backend responds on port 8080.
- Data persists in named volume across container restarts.

VERIFICATION & TEST COVERAGE STRATEGY

- docker build validation
- compose startup verification
- healthcheck validation
- volume persistence test

TASK SPECIFICATION: P1-06 PHASE P1 — FOUNDATIONS & DEVOPS INFRASTRUCTURE

Continuous Integration Pipeline (GitHub Actions CI)

Git Branch: `ci/P1-06-github-actions` · Target Pull Request: PR (ci/P1-06-github-actions)

Core Objective & Technical Scope:
Establish enterprise CI workflow: Automated frontend quality checks, backend Maven verification, Docker builds, and compose smoke testing with PR template and Dependabot.

Priority Level	High	Complexity Rating	`M`
Execution Status	Done ✅	Target PR / Commit	`PR (ci/P1-06-github-actions)`
Upstream Dependencies	P1-05		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author frontend CI job (npm ci, lint, typecheck, vitest, build).
2. Author backend CI job (Java 21 setup, ArchUnit tests, unit tests, JAR packaging).
3. Author Docker verification job validating compose configuration and building test images.
4. Configure standardized GitHub Pull Request template and automated Dependabot scans.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `.github/workflows/ci.yml`
- `.github/pull_request_template.md`
- `.github/dependabot.yml`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- All CI pipeline jobs execute and report green status on PRs.
- Failing unit or architecture tests block merge automatically.

VERIFICATION & TEST COVERAGE STRATEGY

- GitHub Actions automated CI runner verification

TASK SPECIFICATION: P2-01 PHASE P2 — IDENTITY, SECURITY & CAMPUS CORE

Campus Core Schema (University Tenant, College, Building, Term)

Git Branch: `feature/P2-01-campus-schema` · Target Pull Request: PR (feature/P2-01-campus-schema)

Core Objective & Technical Scope:

Database schema and JPA entities for institutional tenancy: Universities, colleges, buildings, and academic terms with strict university_id multi-tenancy enforcement.

Priority Level	High	Complexity Rating	`M`
Execution Status	Done ✅	Target PR / Commit	`PR (feature/P2-01-campus-schema)`
Upstream Dependencies	P1-04, ADR-0003		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author V2__campus.sql migration creating university, college, building, and academic_term tables.
2. Implement BaseTenantEntity providing audit timestamps and tenant partitioning.
3. Create JPA entity models and Spring Data JPA repository interfaces.
4. Implement repository integration tests with Testcontainers verifying constraints.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/src/main/resources/db/migration/V2__campus.sql`
- `campus/domain/*`
- `campus/infra/*Repository.java`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Tables and foreign keys created with unique institutional code constraints.
- Repository tests pass against containerized PostgreSQL.

VERIFICATION & TEST COVERAGE STRATEGY

- CampusRepositoryIntegrationTest
- Flyway migration test

TASK SPECIFICATION: P2-02 PHASE P2 — IDENTITY, SECURITY & CAMPUS CORE

Identity, Security Roles & Vehicles Domain Schema

Git Branch: `feature/P2-02-users-schema` · Target Pull Request: PR (feature/P2-02-users-schema)

Core Objective & Technical Scope:
User account management, security roles (RBAC), vehicle registrations, and refresh token tables with BCrypt hashing and Saudi license plate validation.

Priority Level	High	Complexity Rating	`M`
Execution Status	Done ✅	Target PR / Commit	`PR (feature/P2-02-users-schema)`
Upstream Dependencies	P2-01		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author V3__identity.sql migration creating app_user, user_role, vehicle, and refresh_token tables.
2. Implement User, Role, and Vehicle domain entities with bidirectional relationship mappings.
3. Create SaudiPlate value object with strict regex validation (3 Arabic letters + 1-4 digits).
4. Implement PasswordService using BCrypt with cost factor 12.
5. Write exhaustive unit tests for SaudiPlate parsing and PasswordService hashing.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/src/main/resources/db/migration/V3__identity.sql`
- `identity/domain/*`
- `identity/infra/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- University user ID and Saudi license plate unique per university tenant.
- BCrypt salted password hashes verified; invalid plates rejected.

VERIFICATION & TEST COVERAGE STRATEGY

- SaudiPlateTest
- PasswordServiceTest
- IdentityRepositoryIntegrationTest

TASK SPECIFICATION: P2-03 PHASE P2 — IDENTITY, SECURITY & CAMPUS CORE

Authentication API (Register, Login, Refresh, Logout) & RBAC

Git Branch: `feature/P2-03-authentication` · Target Pull Request: PR (feature/P2-03-authentication)

Core Objective & Technical Scope:
Stateless JWT authentication engine with 15-minute access tokens, rotating refresh tokens, role-based authorization guards, and endpoint rate limiting.

Priority Level	Critical	Complexity Rating	L
Execution Status	Done ✅	Target PR / Commit	`PR (feature/P2-03-authentication)`
Upstream Dependencies	P2-02		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Configure Spring Security with stateless session management and JWT authentication filter.
2. Implement JwtService supporting HMAC-SHA256 signature verification and expiration handling.
3. Implement AuthService handling registration, credential validation, token rotation, and revocation.
4. Create AuthController exposing /api/v1/auth/ endpoints with OpenAPI documentation.
5. Author security test suite verifying authentication matrix and token tampering prevention.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `common/security/*`
- `identity/api/AuthController.java`
- `identity/application/AuthService.java`
- `DTOs`
- `tests`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Registration creates user and registers initial vehicle atomically.
- Invalid credentials return 401 Unauthorized without user enumeration.
- Expired or tampered JWTs rejected immediately; refresh rotation revokes previous tokens.

VERIFICATION & TEST COVERAGE STRATEGY

- AuthServiceTest
- AuthControllerIntegrationTest
- SecurityMatrixTest
- TokenTamperTest

TASK SPECIFICATION: P2-04 PHASE P2 — IDENTITY, SECURITY & CAMPUS CORE

User Profile & Vehicle Registration Management API

Git Branch: `feature/P2-04-profile-vehicles-api` · Target Pull Request: feature/P2-04-profile-vehicles-api

Core Objective & Technical Scope:
Self-service API enabling authenticated students and faculty to retrieve and update profiles and register, update, and manage campus vehicles with IDOR protection.

Priority Level	Medium	Complexity Rating	`M`
Execution Status	In Progress ⏳	Target PR / Commit	`feature/P2-04-profile-vehicles-api`
Upstream Dependencies	P2-03		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Implement ProfileService and VehicleService application logic.
2. Create MeController and VehicleController with DTO request/response contracts.
3. Enforce security ownership checks ensuring users can access only their own vehicles.
4. Author comprehensive unit and MockMvc integration tests.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `identity/api/MeController.java`
- `identity/application/*`
- `DTOs`
- `tests`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Users can view and edit only their own profile and vehicles (zero IDOR vulnerability).
- Validation errors emit standard RFC 7807 problem details.

VERIFICATION & TEST COVERAGE STRATEGY

- ProfileServiceTest
- VehicleControllerTest
- IdorSecurityTest

TASK SPECIFICATION: P2-05 PHASE P2 — IDENTITY, SECURITY & CAMPUS CORE

Development & Demonstration Seed Data Fixture

Git Branch: `feature/P2-05-dev-seed` · Target Pull Request: feature/P2-05-dev-seed

Core Objective & Technical Scope:
Idempotent seed data loader providing realistic FBSU colleges, buildings, parking spots, faculty, and student personas exclusively in dev profile.

Priority Level	Low	Complexity Rating	S
Execution Status	Backlog 📋	Target PR / Commit	`feature/P2-05-dev-seed`
Upstream Dependencies	P2-03		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Implement DevSeeder component activated strictly under @Profile("dev").
2. Ensure idempotent upsert logic to prevent duplicate records across multiple boots.
3. Validate that seeding is completely bypassed in production profile.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/.../DevSeeder.java`
- `seed resources`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Executes seamlessly in dev profile without errors.
- Multiple application restarts produce identical, deduplicated database state.

VERIFICATION & TEST COVERAGE STRATEGY

- DevSeederIntegrationTest verifying profile isolation

TASK SPECIFICATION: P3-01 PHASE P3 — FRONTEND ARCHITECTURE (REACT 19 + TYPESCRIPT)

Design Tokens, Global CSS & Application Shell

Git Branch: `feature/P3-01-app-shell` · Target Pull Request: feature/P3-01-app-shell

Core Objective & Technical Scope:
Port design tokens, brand fonts, and styling into React AppShell with Header, Navigation, Footer, and Router matching the master Figma showcase.

Priority Level	High	Complexity Rating	L
Execution Status	Backlog 📋	Target PR / Commit	`feature/P3-01-app-shell`
Upstream Dependencies	P1-02		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Migrate CSS variable tokens (--fbsu-teal, --fbsu-gold) and fonts into React styles.
2. Build responsive AppShell, TopNav, Dropdowns, and Institutional Footer.
3. Configure React Router with core layout routes and error boundaries.
4. Capture visual regression screenshot baselines across desktop and mobile viewports.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/styles/*`
- `frontend/src/app/*`
- `frontend/src/shared/layout/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Visual parity confirmed against master design showcase in both dark and light modes.
- Zero layout shifts across desktop, tablet, and mobile displays.

VERIFICATION & TEST COVERAGE STRATEGY

- Component unit tests
- Playwright responsive visual verification

TASK SPECIFICATION: P3-02 PHASE P3 — FRONTEND ARCHITECTURE (REACT 19 + TYPESCRIPT)

Internationalization (AR/EN), RTL/LTR Switching & Theme Toggle

Git Branch: `feature/P3-02-i18n-theme` · Target Pull Request: feature/P3-02-i18n-theme

Core Objective & Technical Scope:
Bilingual internationalization engine (Arabic/English) via react-i18next with bidirectional RTL/LTR layout adaptation and persisted theme toggle.

Priority Level	High	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P3-02-i18n-theme`
Upstream Dependencies	P3-01		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Extract all interface strings into structured ar.json and en.json translation catalogs.
2. Configure I18nextProvider with dynamic document dir and lang attribute updates.
3. Implement ThemeToggle (dark/light) with local storage persistence.
4. Write tests verifying dynamic language and direction switching.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/shared/i18n/*`
- `locales/ar.json`
- `locales/en.json`
- `ThemeToggle/LangToggle`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Zero hardcoded strings in application shell.
- Document direction toggles between rtl and ltr seamlessly without broken layouts.

VERIFICATION & TEST COVERAGE STRATEGY

- I18n unit tests
- Playwright bilingual switching test

TASK SPECIFICATION: P3-03 PHASE P3 — FRONTEND ARCHITECTURE (REACT 19 + TYPESCRIPT)

Typed API Client, Auth Context & Route Guards

Git Branch: `feature/P3-03-api-client-auth-context` · Target Pull Request: feature/P3-03-api-client-auth-context

Core Objective & Technical Scope:
Typed HTTP client with Axios, TanStack Query integration, AuthContext provider, automated JWT token refresh interceptors, and protected route guards.

Priority Level	High	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P3-03-api-client-auth-context`
Upstream Dependencies	P3-01, P2-03		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Create configured Axios client with automatic Authorization header injection.
2. Implement token refresh interceptor on 401 responses with queued request replay.
3. Build AuthProvider managing user session, roles, and login/logout state.
4. Implement ProtectedRoute and RoleGuard components for declarative authorization.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/shared/http/*`
- `frontend/src/features/auth/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- 401 response triggers token refresh once then logs out on failure.
- Unauthorized access attempts redirect cleanly to login with return path preserved.

VERIFICATION & TEST COVERAGE STRATEGY

- HTTP interceptor tests
- MSW mock integration tests
- Route guard tests

TASK SPECIFICATION: P3-04 PHASE P3 — FRONTEND ARCHITECTURE (REACT 19 + TYPESCRIPT)

Authentication UI (Login & Registration Flow)

Git Branch: `feature/P3-04-auth-ui` · Target Pull Request: feature/P3-04-auth-ui

Core Objective & Technical Scope:
User-friendly login and registration forms featuring Zod schema validation, API wiring, error notifications, and removal of quick-login development bypasses.

Priority Level	High	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P3-04-auth-ui`
Upstream Dependencies	P3-03, P2-03		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Build LoginForm and RegisterForm components matching Figma specifications.
2. Implement client-side validation using Zod and React Hook Form.
3. Wire forms to AuthService and handle server error feedback.
4. Ensure accessible keyboard navigation and ARIA attributes.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/features/auth/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Instant field-level validation feedback on invalid inputs.
- Successful login/registration persists authentication session and redirects.

VERIFICATION & TEST COVERAGE STRATEGY

- Form validation tests
- Auth API integration tests
- Playwright login flow

TASK SPECIFICATION: P3-05 PHASE P3 — FRONTEND ARCHITECTURE (REACT 19 + TYPESCRIPT)

Shared UI Component Kit (Modal, Toast, Badge, Button, SaudiPlate, QR)

Git Branch: `feature/P3-05-ui-kit` · Target Pull Request: feature/P3-05-ui-kit

Core Objective & Technical Scope:
Reusable atomic component library: Modal with focus-trap, Toast notification manager, Badge, Button, FormField, SaudiPlate renderer, and QR code generator.

Priority Level	Medium	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P3-05-ui-kit`
Upstream Dependencies	P3-01		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Implement atomic components matching design system token definitions.
2. Implement accessible Modal component with focus trap and escape key listeners.
3. Implement SaudiPlate component rendering official Saudi plate layout with Arabic lettering.
4. Perform automated accessibility audit using axe-core.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/shared/ui/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- All interactive components fully operable via keyboard.
- Zero accessibility violations reported by axe-core.

VERIFICATION & TEST COVERAGE STRATEGY

- Component unit tests
- Axe accessibility compliance tests

TASK SPECIFICATION: P4-01 PHASE P4 — FINANCIAL LEDGER & TUITION OFFSET ENGINE

Financial Ledger & Wallet Database Schema

Git Branch: `feature/P4-01-ledger-schema` · Target Pull Request: feature/P4-01-ledger-schema

Core Objective & Technical Scope:
Append-only double-entry financial ledger schema and student wallet tables with mathematical constraints guaranteeing non-negative balances.

Priority Level	High	Complexity Rating	M
Execution Status	Backlog 📋	Target PR / Commit	`feature/P4-01-ledger-schema`
Upstream Dependencies	P2-03		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author V4__wallet.sql migration creating wallet and ledger_entry tables.
2. Apply CHECK (balance >= 0) constraint on wallet balances.
3. Implement optimistic locking (@Version) on Wallet entity to prevent race conditions.
4. Create WalletRepository and LedgerEntryRepository interfaces.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/src/main/resources/db/migration/V4__wallet.sql`
- `wallet/domain/*`
- `wallet/infra/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Negative balance operations physically rejected by PostgreSQL engine.
- Idempotency key uniqueness constraint enforced on all financial transactions.

VERIFICATION & TEST COVERAGE STRATEGY

- WalletRepositoryIntegrationTest
- PostgreSQL constraint validation tests

TASK SPECIFICATION: P4-02 PHASE P4 — FINANCIAL LEDGER & TUITION OFFSET ENGINE

Wallet Service & Transaction Operations API

Git Branch: `feature/P4-02-wallet-api` · Target Pull Request: feature/P4-02-wallet-api

Core Objective & Technical Scope:
Enterprise financial engine: Real-time balance inquiry, paginated transaction statement, atomic peer-to-peer transfers, and idempotent payment handling.

Priority Level	Critical	Complexity Rating	L
Execution Status	Backlog 📋	Target PR / Commit	`feature/P4-02-wallet-api`
Upstream Dependencies	P4-01		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Implement WalletService supporting credit, debit, and atomic transfer methods.
2. Implement Idempotency-Key header filter guaranteeing safe retry semantics.
3. Create WalletController exposing /api/v1/wallet/ endpoints with OpenAPI documentation.
4. Execute multi-threaded stress tests to verify race condition resilience.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `wallet/application/*`
- `wallet/api/*`
- `tests`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Peer-to-peer transfers execute atomically inside a single database transaction.
- Zero double-spending under concurrent multi-threaded requests.

VERIFICATION & TEST COVERAGE STRATEGY

- WalletServiceTest
- ConcurrentTransferStressTest
- WalletControllerTest

TASK SPECIFICATION: P4-03 PHASE P4 — FINANCIAL LEDGER & TUITION OFFSET ENGINE

Tuition Account Management & Automatic Fee Offset API

Git Branch: `feature/P4-03-tuition` · Target Pull Request: feature/P4-03-tuition

Core Objective & Technical Scope:
Academic term tuition invoice accounts, automated credit offset from peer tutoring earnings, and unique fiscal receipt number generation.

Priority Level	High	Complexity Rating	L
Execution Status	Backlog 📋	Target PR / Commit	`feature/P4-03-tuition`
Upstream Dependencies	P4-02		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author V5__tuition.sql migration creating tuition_account and payment_receipt tables.
2. Implement TuitionService executing atomic tuition payment via wallet balance offset.
3. Implement sequential institutional receipt number generator.
4. Create TuitionController and DTOs with OpenAPI documentation.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/src/main/resources/db/migration/V5__tuition.sql`
- `wallet/application/Tuition*`
- `wallet/api/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Offset amount cannot exceed available wallet balance or remaining invoice due.
- Receipt numbers are globally unique, sequential, and immutable.

VERIFICATION & TEST COVERAGE STRATEGY

- TuitionServiceTest
- ReceiptGenerationTest
- TuitionControllerTest

TASK SPECIFICATION: P4-04 PHASE P4 — FINANCIAL LEDGER & TUITION OFFSET ENGINE

Student Wallet UI Dashboard

Git Branch: `feature/P4-04-wallet-ui` · Target Pull Request: feature/P4-04-wallet-ui

Core Objective & Technical Scope:
Interactive wallet interface presenting current SAR balance, categorized transaction ledger history, and tutoring earnings breakdown.

Priority Level	Medium	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P4-04-wallet-ui`
Upstream Dependencies	P3-04, P4-02		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Build WalletDashboard with balance card, quick actions, and filterable history table.
2. Wire components to Wallet API using TanStack Query.
3. Implement skeleton loading states and empty state indicators.
4. Verify responsiveness on mobile and desktop viewports.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/features/wallet/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Exact visual parity with master Figma design system.
- Real-time balance updates accurately reflect server state.

VERIFICATION & TEST COVERAGE STRATEGY

- WalletDashboard component tests
- E2E wallet transaction flow

TASK SPECIFICATION: P4-05 PHASE P4 — FINANCIAL LEDGER & TUITION OFFSET ENGINE

Tuition Payment & Digital Receipt UI

Git Branch: `feature/P4-05-tuition-ui` · Target Pull Request: feature/P4-05-tuition-ui

Core Objective & Technical Scope:
Tuition fee payment interface, credit offset modal, and printable official institutional payment receipt with validation QR code.

Priority Level	Medium	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P4-05-tuition-ui`
Upstream Dependencies	P4-03, P4-04		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Build TuitionOverview displaying current term fees and pending balance.
2. Implement payment modal allowing partial or full offset from wallet earnings.
3. Build printable official institutional receipt view with formatted print CSS.
4. Test complete payment flow end-to-end.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/features/tuition/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Accurate balance calculation preventing invalid payment attempts.
- Receipt prints cleanly on A4 with university header and validation code.

VERIFICATION & TEST COVERAGE STRATEGY

- TuitionForm validation tests
- Receipt print CSS test
- Playwright payment flow

TASK SPECIFICATION: P5-01 PHASE P5 — SMART PARKING & ALPR COMPUTER VISION GATE

Parking Domain Schema (Zones, Spots, Assignments, Tariffs)

Git Branch: `feature/P5-01-parking-schema` · Target Pull Request: feature/P5-01-parking-schema

Core Objective & Technical Scope:

Database schema for campus parking zones, 30 monitored spots, vehicle permits, and dynamic hourly parking tariff schedules.

Priority Level	High	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P5-01-parking-schema`
Upstream Dependencies	P2-01		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author V6__parking.sql migration creating parking_zone, parking_spot, and tariff tables.
2. Create Zone, Spot, and Tariff JPA entities and repositories.
3. Seed baseline definitions for 30 monitored campus spots across North and South zones.
4. Validate foreign keys and constraints.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/src/main/resources/db/migration/V6__parking.sql`
- `parking/domain/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Spot numbering unique per zone.
- Tariff schedules support dynamic hourly rates and faculty exemption rules.

VERIFICATION & TEST COVERAGE STRATEGY

- ParkingRepositoryTest
- Flyway migration test

TASK SPECIFICATION: P5-02 PHASE P5 — SMART PARKING & ALPR COMPUTER VISION GATE

Parking Spots & Zones Real-Time Query API

Git Branch: `feature/P5-02-spots-api` · Target Pull Request: feature/P5-02-spots-api

Core Objective & Technical Scope:
High-performance read API delivering real-time spot occupancy status with zone, status, and accessibility filtering.

Priority Level	High	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P5-02-spots-api`
Upstream Dependencies	P5-01		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Implement SpotService with optimized projection queries for rapid grid rendering.
2. Create SpotController exposing /api/v1/parking/spots with filter parameters.
3. Implement zone aggregate statistics endpoint (total, available, occupied).
4. Author API integration tests.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `parking/api/*`
- `parking/application/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Accurate status filtering (Available, Occupied, Reserved, Maintenance).
- Sub-50ms API response time under concurrent read load.

VERIFICATION & TEST COVERAGE STRATEGY

- SpotServiceTest
- SpotControllerMockMvcTest

TASK SPECIFICATION: P5-03 PHASE P5 — SMART PARKING & ALPR COMPUTER VISION GATE

Parking Reservation Engine & Digital Pass Generation

Git Branch: `feature/P5-03-parking-booking` · Target Pull Request: feature/P5-03-parking-booking

Core Objective & Technical Scope:
Collision-free spot reservation backed by PostgreSQL GiST exclusion constraints, server-side tariff calculation, and encrypted QR pass.

Priority Level	Critical	Complexity Rating	L
Execution Status	Backlog 📋	Target PR / Commit	`feature/P5-03-parking-booking`
Upstream Dependencies	P5-02, P4-02		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author V7__booking.sql migration applying btree_gist exclusion constraint on (spot_id, booking_interval).
2. Implement PricingService computing exact tariff based on vehicle type and duration.
3. Wire reservation creation with atomic wallet debit in single transaction.
4. Generate encrypted pass payload with QR verification token.
5. Execute multi-threaded concurrency tests to verify collision prevention.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/src/main/resources/db/migration/V7__booking.sql`
- `parking/application/Booking*`
- `parking/api/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Double-booking mathematically impossible under concurrent execution.
- Payment failure rolls back reservation atomically.

VERIFICATION & TEST COVERAGE STRATEGY

- PricingServiceTest
- ConcurrencyBookingStressTest
- BookingControllerTest

TASK SPECIFICATION: P5-04 PHASE P5 — SMART PARKING & ALPR COMPUTER VISION GATE

Flex-Share Peer Parking Exchange & Rewards Engine

Git Branch: `feature/P5-04-flex-share` · Target Pull Request: feature/P5-04-flex-share

Core Objective & Technical Scope:
Intelligent spot-sharing marketplace allowing permit holders to temporarily release unused spots to peers during classes and earn wallet rewards.

Priority Level	High	Complexity Rating	L
Execution Status	Backlog 📋	Target PR / Commit	`feature/P5-04-flex-share`
Upstream Dependencies	P5-03		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Create FlexShareOffer and FlexShareClaim domain entities.
2. Implement slot matching service verifying return-time windows.
3. Implement idempotent reward credit service updating owner wallet upon release.
4. Create FlexShareController and abuse prevention guards.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `parking/domain/Flex*`
- `parking/application/Flex*`
- `parking/api/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Reward credited exactly once per verified exchange.
- Spot guaranteed available for primary permit holder upon scheduled return.

VERIFICATION & TEST COVERAGE STRATEGY

- FlexShareServiceTest
- ConcurrentClaimTest
- FlexShareControllerTest

TASK SPECIFICATION: P5-05 PHASE P5 — SMART PARKING & ALPR COMPUTER VISION GATE

ALPR Gate Access Decision Engine (Hardware Adapter Port)

Git Branch: `feature/P5-05-gate-api` · Target Pull Request: feature/P5-05-gate-api

Core Objective & Technical Scope:
Low-latency webhook API processing vehicle license plates, verifying active permits, and actuating gate barriers via a pluggable adapter.

Priority Level	Medium	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P5-05-gate-api`
Upstream Dependencies	P5-03		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Define GateHardwarePort interface and implement SimulatedAlprAdapter.
2. Implement GateDecisionService resolving license plates against active bookings.
3. Log 100% of gate entry/exit events in gate_event audit table.
4. Expose /api/v1/gate/scan webhook endpoint with sub-200ms target latency.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `parking/application/Gate*`
- `parking/infra/SimulatedAlprAdapter.java`
- `parking/api/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Unambiguous access decision (ALLOW/DENY) with explicit reason code for every scan.
- Audit log tracks timestamp, plate, confidence score, and decision.

VERIFICATION & TEST COVERAGE STRATEGY

- GateDecisionServiceTest
- GateControllerTest
- PerformanceLatencyTest

TASK SPECIFICATION: P5-06 PHASE P5 — SMART PARKING & ALPR COMPUTER VISION GATE

Interactive 30-Spot Parking Grid UI

Git Branch: `feature/P5-06-map-ui` · Target Pull Request: feature/P5-06-map-ui

Core Objective & Technical Scope:
Visual parking map rendering all 30 campus spots with live status badges, zone filters, and spot detail dialogs.

Priority Level	High	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P5-06-map-ui`
Upstream Dependencies	P3-05, P5-02		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Build SpotCard and ParkingGrid components with color-coded status badges.
2. Implement zone filter tabs (All, North Zone, South Zone, Disabled).
3. Build SpotDetailModal displaying spot telemetry and reservation button.
4. Verify responsiveness and touch targets on mobile viewports.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/features/parking/map/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Visual parity with master design showcase in both dark and light modes.
- Grid renders smoothly with zero layout jumps upon status changes.

VERIFICATION & TEST COVERAGE STRATEGY

- ParkingGrid component tests
- SpotCard test
- Playwright map interaction

TASK SPECIFICATION: P5-07 PHASE P5 — SMART PARKING & ALPR COMPUTER VISION GATE

Parking Booking & Encrypted Pass Modal UI

Git Branch: `feature/P5-07-booking-ui` · Target Pull Request: feature/P5-07-booking-ui

Core Objective & Technical Scope:
Reservation modal with duration sliders, vehicle picker, live tariff calculation, and scannable QR gate pass display.

Priority Level	High	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P5-07-booking-ui`
Upstream Dependencies	P5-03, P5-06		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Build BookingModal with duration slider, vehicle selection, and live cost calculation.
2. Wire reservation submission to Booking API with optimistic UI updates.
3. Build DigitalPassModal rendering encrypted QR code and booking reference.
4. Handle error cases (insufficient funds, slot conflict).

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/features/parking/booking/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Calculated price matches server-side tariff calculation exactly.
- QR pass renders crisply and scans cleanly with standard barcode scanners.

VERIFICATION & TEST COVERAGE STRATEGY

- BookingModal validation test
- PassModal QR test
- Playwright booking flow

TASK SPECIFICATION: P5-08 PHASE P5 — SMART PARKING & ALPR COMPUTER VISION GATE

Flex-Share Marketplace UI

Git Branch: `feature/P5-08-flex-ui` · Target Pull Request: feature/P5-08-flex-ui

Core Objective & Technical Scope:
One-click spot sharing activation toggle, projected reward estimator, and active exchange status tracker.

Priority Level	Medium	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P5-08-flex-ui`
Upstream Dependencies	P5-04, P5-06		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Build FlexShareToggle and reward estimation widget.
2. Build active exchange status banner and claim confirmation dialog.
3. Wire components to Flex-Share API endpoints.
4. Handle expiration and cancellation states.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/features/parking/flex/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Exact visual match with master design showcase.
- Prevents duplicate offer activation on already shared spots.

VERIFICATION & TEST COVERAGE STRATEGY

- FlexShare component tests
- Playwright exchange flow

TASK SPECIFICATION: P5-09 PHASE P5 — SMART PARKING & ALPR COMPUTER VISION GATE

ALPR Camera Scanner & IoT Sensor Simulator UI

Git Branch: `feature/P5-09-gate-sim-ui` · Target Pull Request: feature/P5-09-gate-sim-ui

Core Objective & Technical Scope:
Interactive presentation simulator demonstrating gate camera OCR recognition, distance slider sensor injection, and hardware diagnostics.

Priority Level	Low	Complexity Rating	M
Execution Status	Backlog 📋	Target PR / Commit	`feature/P5-09-gate-sim-ui`
Upstream Dependencies	P5-05		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Build GateScannerPreview demonstrating OCR plate framing and barrier animation.
2. Build SimulatorPanel with ultrasonic sensor distance sliders and event triggers.
3. Integrate subtle audio feedback synthesizer for successful badge scans.
4. Wire simulator events to backend gate decision endpoint.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/features/parking/gate/*`
- `simulator/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Demonstrates full entry/exit pipeline with realistic visual telemetry.
- Audio synthesizer plays feedback without blocking the UI thread.

VERIFICATION & TEST COVERAGE STRATEGY

- GateScanner component tests
- Simulator interaction tests

TASK SPECIFICATION: P6-01 PHASE P6 — ACADEMIC CAMPUS SERVICES & RESOURCE PODS

Peer Tutoring Course Catalog & Offerings API

Git Branch: `feature/P6-01-tutoring-api` · Target Pull Request: feature/P6-01-tutoring-api

Core Objective & Technical Scope:
Certified course catalog, tutor qualification verification, and session offering publishing API.

Priority Level	High	Complexity Rating	L
Execution Status	Backlog 📋	Target PR / Commit	`feature/P6-01-tutoring-api`
Upstream Dependencies	P2-03		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author V8__tutoring.sql migration creating course and tutoring_session tables.
2. Implement Course and TutoringSession JPA domain entities.
3. Implement TutoringService with category, course, and date filters.
4. Create TutoringController exposing /api/v1/tutoring/ endpoints.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/src/main/resources/db/migration/V8__tutoring.sql`
- `tutoring/**`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Only qualified honors students permitted to publish tutoring sessions.
- Search query supports course code, title, and tutor name.

VERIFICATION & TEST COVERAGE STRATEGY

- TutoringServiceTest
- TutoringRepositoryTest
- TutoringControllerTest

TASK SPECIFICATION: P6-02 PHASE P6 — ACADEMIC CAMPUS SERVICES & RESOURCE PODS

Tutoring Booking & Automated 70 SAR/Hour Escrow Transfer

Git Branch: `feature/P6-02-tutoring-booking` · Target Pull Request: feature/P6-02-tutoring-booking

Core Objective & Technical Scope:
Session enrollment engine executing atomic fee transfers (70 SAR/hr standard rate) from student wallet to tutor tuition credit account.

Priority Level	Critical	Complexity Rating	L
Execution Status	Backlog 📋	Target PR / Commit	`feature/P6-02-tutoring-booking`
Upstream Dependencies	P6-01, P4-02		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Implement enrollment service enforcing student capacity limits.
2. Execute atomic wallet transfer deducting 70 SAR/hr and crediting tutor ledger.
3. Prevent duplicate enrollment and tutor self-booking.
4. Expose booking endpoint with confirmation notifications.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `tutoring/application/*`
- `tutoring/api/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Insufficient wallet balance blocks booking with clear error message.
- Standardized 70 SAR/hr rate enforced strictly on backend.

VERIFICATION & TEST COVERAGE STRATEGY

- TutoringBookingServiceTest
- ConcurrentEnrollmentTest
- BookingControllerTest

TASK SPECIFICATION: P6-03 PHASE P6 — ACADEMIC CAMPUS SERVICES & RESOURCE PODS

Peer Tutoring Marketplace UI

Git Branch: `feature/P6-03-tutoring-ui` · Target Pull Request: feature/P6-03-tutoring-ui

Core Objective & Technical Scope:
Student tutoring portal with course filtering, tutor rating cards, booking dialog, and tutor session creation modal.

Priority Level	High	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P6-03-tutoring-ui`
Upstream Dependencies	P6-02, P3-05		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Build TutoringHub with subject filters and course search bar.
2. Build TutorCard displaying verified badge, hourly rate (70 SAR), and availability.
3. Build BookingModal showing session details and wallet balance confirmation.
4. Build CreateSessionModal for approved student tutors.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/features/tutoring/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Visual parity with master design showcase in dark and light modes.
- Instant wallet balance debit confirmation feedback.

VERIFICATION & TEST COVERAGE STRATEGY

- TutoringHub component tests
- BookingModal validation test
- Playwright tutoring flow

TASK SPECIFICATION: P6-04 PHASE P6 — ACADEMIC CAMPUS SERVICES & RESOURCE PODS

Faculty Office Hours & Dean's Hub API

Git Branch: `feature/P6-04-office-hours-api` · Target Pull Request: feature/P6-04-office-hours-api

Core Objective & Technical Scope:
College leadership and faculty availability windows management with collision-free student appointment booking.

Priority Level	High	Complexity Rating	L
Execution Status	Backlog 📋	Target PR / Commit	`feature/P6-04-office-hours-api`
Upstream Dependencies	P2-03		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author V9__office_hours.sql migration with btree_gist exclusion on faculty appointment intervals.
2. Implement FacultyWindow and Appointment domain entities.
3. Implement OfficeHoursService managing recurring windows and appointment bookings.
4. Create OfficeHoursController with topic classification (Senior Project, Logic Design, Advising).

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/src/main/resources/db/migration/V9__office_hours.sql`
- `officehours/**`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Overlapping appointments physically impossible under database exclusion constraints.
- Faculty members can configure and manage only their own office hours.

VERIFICATION & TEST COVERAGE STRATEGY

- OfficeHoursServiceTest
- AppointmentExclusionConstraintTest
- OfficeHoursControllerTest

TASK SPECIFICATION: P6-05 PHASE P6 — ACADEMIC CAMPUS SERVICES & RESOURCE PODS

Faculty Office Hours & Dean's Appointment UI

Git Branch: `feature/P6-05-office-hours-ui` · Target Pull Request: feature/P6-05-office-hours-ui

Core Objective & Technical Scope:
Faculty directory cards (Dean of Computing, Department Chairs), available time slot picker, and appointment booking confirmation.

Priority Level	Medium	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P6-05-office-hours-ui`
Upstream Dependencies	P6-04, P3-05		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Build FacultyDirectory listing academic leaders and professors.
2. Build SlotPicker displaying real-time available appointment slots.
3. Build AppointmentModal supporting topic selection and notes.
4. Wire appointment confirmation with calendar export.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/features/office-hours/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Visual match with Figma design system.
- Unavailable or booked slots disabled dynamically.

VERIFICATION & TEST COVERAGE STRATEGY

- FacultyDirectory component tests
- AppointmentModal test
- Playwright booking flow

TASK SPECIFICATION: P6-06 PHASE P6 — ACADEMIC CAMPUS SERVICES & RESOURCE PODS

Study Rooms & Computer Labs Reservation API

Git Branch: `feature/P6-06-rooms-api` · Target Pull Request: feature/P6-06-rooms-api

Core Objective & Technical Scope:
Collaborative study pods and laboratory booking engine with active session extension, release, and GiST overlap prevention.

Priority Level	High	Complexity Rating	L
Execution Status	Backlog 📋	Target PR / Commit	`feature/P6-06-rooms-api`
Upstream Dependencies	P2-03		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author V10__rooms.sql migration with btree_gist exclusion on room booking intervals.
2. Implement Room and RoomBooking domain entities.
3. Implement RoomService supporting create, extend, and release operations.
4. Enforce maximum consecutive booking duration limits.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/src/main/resources/db/migration/V10__rooms.sql`
- `rooms/**`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Collision-free scheduling guaranteed across all study pods and labs.
- Early release immediately frees slot for other waiting workgroups.

VERIFICATION & TEST COVERAGE STRATEGY

- RoomServiceTest
- RoomBookingExclusionTest
- RoomControllerTest

TASK SPECIFICATION: P6-07 PHASE P6 — ACADEMIC CAMPUS SERVICES & RESOURCE PODS

Study Rooms UI & Digital Evacuation Timer

Git Branch: `feature/P6-07-rooms-ui` · Target Pull Request: feature/P6-07-rooms-ui

Core Objective & Technical Scope:
Room catalog cards, live occupancy indicators, pod booking dialog, and prominent real-time countdown evacuation timer.

Priority Level	Medium	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P6-07-rooms-ui`
Upstream Dependencies	P6-06, P3-05		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Build RoomCatalog displaying pod capacity, equipment tags, and availability status.
2. Build RoomBookingModal supporting team member entry and duration selection.
3. Build EvacuationTimer displaying live countdown with color-coded warning thresholds.
4. Implement extension request and early release actions.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/features/rooms/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Matches master design showcase.
- Countdown timer decrements accurately without memory leaks or drift.

VERIFICATION & TEST COVERAGE STRATEGY

- RoomCatalog component tests
- EvacuationTimer test
- Playwright room reservation flow

TASK SPECIFICATION: P7-01 PHASE P7 — NOTIFICATIONS, GOVERNANCE & TELEMETRY ANALYTICS

In-App Notification Engine & Notification API

Git Branch: `feature/P7-01-notifications-api` · Target Pull Request: feature/P7-01-notifications-api

Core Objective & Technical Scope:

Real-time notification service managing unread alerts, batch marking as read, and persistent event notifications.

Priority Level	High	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P7-01-notifications-api`
Upstream Dependencies	P2-03		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author V11__notifications.sql migration creating in_app_notification table.
2. Implement Notification domain entity, repository, and service.
3. Implement NotificationController with mark-as-read and unread count endpoints.
4. Enforce security checks ensuring users access only their own notifications.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/src/main/resources/db/migration/V11__notifications.sql`
- `notification/**`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Notification queries return only recipient records with zero data leakage.
- Unread count query optimized for frequent polling.

VERIFICATION & TEST COVERAGE STRATEGY

- NotificationServiceTest
- NotificationControllerTest
- SecurityIsolationTest

TASK SPECIFICATION: P7-02 PHASE P7 — NOTIFICATIONS, GOVERNANCE & TELEMETRY ANALYTICS

Event-Driven Notification Bus Dispatcher

Git Branch: `feature/P7-02-notification-events` · Target Pull Request: feature/P7-02-notification-events

Core Objective & Technical Scope:
Decoupled Spring application event listeners automatically triggering notifications for parking reservations, payments, and tutoring sessions.

Priority Level	High	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P7-02-notification-events`
Upstream Dependencies	P7-01, P4-02, P5-03, P6-02, P6-04		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Define domain events (BookingCreatedEvent, PaymentProcessedEvent, SessionBookedEvent).
2. Implement asynchronous @TransactionalEventListener handlers in notification module.
3. Format bilingual notification templates with action links.
4. Write integration tests verifying end-to-end event propagation.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `Domain modules: event publishers`
- `notification/application/listeners/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Every major business transaction produces an in-app notification reliably.
- Event failures do not break the underlying business transaction.

VERIFICATION & TEST COVERAGE STRATEGY

- NotificationEventListenerIntegrationTest

TASK SPECIFICATION: P7-03 PHASE P7 — NOTIFICATIONS, GOVERNANCE & TELEMETRY ANALYTICS

Notification Center UI (Header Bell & Slide-out Drawer)

Git Branch: `feature/P7-03-notifications-ui` · Target Pull Request: feature/P7-03-notifications-ui

Core Objective & Technical Scope:

Top bar notification bell with animated unread badge counter and slide-out notification drawer with instant read actions.

Priority Level	Medium	Complexity Rating	S
Execution Status	Backlog 📋	Target PR / Commit	`feature/P7-03-notifications-ui`
Upstream Dependencies	P7-01		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Build NotificationBell component with animated unread counter.
2. Build NotificationDrawer displaying categorized notification cards.
3. Implement TanStack Query polling and optimistic mark-as-read mutations.
4. Add notification sound feedback.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/features/notifications/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Unread badge count synchronizes accurately with server state.
- Drawer slides smoothly with full keyboard accessibility.

VERIFICATION & TEST COVERAGE STRATEGY

- NotificationBell component tests
- NotificationDrawer interaction test

TASK SPECIFICATION: P7-04 PHASE P7 — NOTIFICATIONS, GOVERNANCE & TELEMETRY ANALYTICS

System Administration & Governance API

Git Branch: `feature/P7-04-admin-api` · Target Pull Request: feature/P7-04-admin-api

Core Objective & Technical Scope:
Privileged administrative endpoints for managing campus parking spots, study rooms, tariff rules, and user role assignments.

Priority Level	Medium	Complexity Rating	L
Execution Status	Backlog 📋	Target PR / Commit	`feature/P7-04-admin-api`
Upstream Dependencies	P5-01, P6-06		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Implement admin controllers with @PreAuthorize("hasRole('CAMPUS_ADMIN')").
2. Provide administrative CRUD endpoints for spots, rooms, and tariffs.
3. Record all administrative overrides in audit log ledger.
4. Write security tests verifying access restriction.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `Admin endpoints across domain modules`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Non-admin requests receive 403 Forbidden with RFC 7807 problem details.
- Every administrative modification produces an audit trail.

VERIFICATION & TEST COVERAGE STRATEGY

- AdminControllerTest
- SecurityMatrixTest
- AuditLoggingTest

TASK SPECIFICATION: P7-05 PHASE P7 — NOTIFICATIONS, GOVERNANCE & TELEMETRY ANALYTICS

Administrative Control Dashboard UI

Git Branch: `feature/P7-05-admin-ui` · Target Pull Request: feature/P7-05-admin-ui

Core Objective & Technical Scope:
Administrative back-office management tables for campus resource configuration, user roles, and spot status overrides.

Priority Level	Medium	Complexity Rating	L
Execution Status	Backlog 📋	Target PR / Commit	`feature/P7-05-admin-ui`
Upstream Dependencies	P7-04		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Build AdminLayout with admin navigation guard.
2. Build SpotManagementTable with status override actions.
3. Build UserManagementTable with role assignment controls.
4. Wire forms and modal dialogs to Admin API.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/src/features/admin/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Completely inaccessible to non-admin roles.
- Modifications update data tables immediately with toast confirmations.

VERIFICATION & TEST COVERAGE STRATEGY

- AdminLayout guard tests
- SpotManagementTable test
- Playwright admin flow

TASK SPECIFICATION: P7-06 PHASE P7 — NOTIFICATIONS, GOVERNANCE & TELEMETRY ANALYTICS

Campus Analytics & Reporting Telemetry Engine

Git Branch: `feature/P7-06-analytics` · Target Pull Request: feature/P7-06-analytics

Core Objective & Technical Scope:
Real-time analytics queries calculating hourly spot utilization, peak arrival curves, tutoring volume, and carbon emission offsets.

Priority Level	Low	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`feature/P7-06-analytics`
Upstream Dependencies	P5-03		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author optimized SQL analytical aggregation queries.
2. Create AnalyticsController exposing metrics endpoints.
3. Build interactive chart visualizations in frontend (peak hours, occupancy curves).
4. Verify performance under large historical datasets.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `analytics/**`
- `frontend/src/features/analytics/*`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- All analytics metrics derived from actual database records with zero synthetic mock data.
- Aggregation queries execute in under 100ms.

VERIFICATION & TEST COVERAGE STRATEGY

- AnalyticsRepositoryTest
- AnalyticsControllerTest
- ChartComponentTest

TASK SPECIFICATION: P8-01 PHASE P8 — SECURITY HARDENING, COMPLIANCE & QUALITY ASSURANCE

Comprehensive Security Audit (OWASP ASVS, Dependency Scan & Headers)

Git Branch: `test/P8-01-security-review` · Target Pull Request: test/P8-01-security-review

Core Objective & Technical Scope:
Security vulnerability assessment following OWASP ASVS guidelines, OWASP dependency vulnerability scanning, and HTTP security header hardening.

Priority Level	High	Complexity Rating	L
Execution Status	Backlog 📋	Target PR / Commit	`test/P8-01-security-review`
Upstream Dependencies	P7-06		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Execute OWASP ASVS Level 2 verification audit.
2. Run automated dependency vulnerability scans across Maven and npm.
3. Harden Content-Security-Policy, HSTS, X-Content-Type-Options, and CORS headers.
4. Author penetration test suite covering injection and broken authorization.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `security configuration`
- `docs/security-review.md`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Zero High or Critical vulnerabilities in application dependencies.
- Security headers present and verified on all HTTP responses.

VERIFICATION & TEST COVERAGE STRATEGY

- DependencyCheck scan
- SecurityHeaderTest
- PenetrationTestSuite

TASK SPECIFICATION: P8-02 PHASE P8 — SECURITY HARDENING, COMPLIANCE & QUALITY ASSURANCE

Comprehensive End-to-End Test Suite (Playwright)

Git Branch: `test/P8-02-e2e-suite` · Target Pull Request: test/P8-02-e2e-suite

Core Objective & Technical Scope:
Automated browser test suite verifying core user journeys across Arabic and English locales and dual light/dark themes.

Priority Level	High	Complexity Rating	L
Execution Status	Backlog 📋	Target PR / Commit	`test/P8-02-e2e-suite`
Upstream Dependencies	P7-06		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Configure Playwright test harness with authentication fixtures.
2. Write E2E test specs for parking booking, tutoring enrollment, and tuition payment.
3. Test bilingual switching (AR/EN) and dark/light theme persistence.
4. Integrate Playwright suite into GitHub Actions CI pipeline.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/e2e/**`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- 100% pass rate across all critical user journeys on Chromium, Firefox, and WebKit.
- Automated video and trace capture on failure.

VERIFICATION & TEST COVERAGE STRATEGY

- Playwright E2E full test suite

TASK SPECIFICATION: P8-03 PHASE P8 — SECURITY HARDENING, COMPLIANCE & QUALITY ASSURANCE

Responsive Design & Accessibility (WCAG 2.1 AA) Audit

Git Branch: `fix/P8-03-responsive-a11y` · Target Pull Request: fix/P8-03-responsive-a11y

Core Objective & Technical Scope:
Multi-device viewport validation (mobile 360px, tablet 768px, desktop 1280px+) and automated axe accessibility auditing.

Priority Level	Medium	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`fix/P8-03-responsive-a11y`
Upstream Dependencies	P7-06		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Run automated axe accessibility scans across all application routes.
2. Audit color contrast ratios and keyboard focus navigation rings.
3. Verify viewport responsiveness at 360px, 768px, 1024px, and 1440px.
4. Fix any identified contrast or keyboard accessibility defects.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `frontend/**`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Zero critical accessibility violations under WCAG 2.1 Level AA standards.
- Zero horizontal overflow or layout breakage across all viewports.

VERIFICATION & TEST COVERAGE STRATEGY

- Axe accessibility audit
- Playwright multi-viewport visual test

TASK SPECIFICATION: P8-04 PHASE P8 — SECURITY HARDENING, COMPLIANCE & QUALITY ASSURANCE

High-Concurrency Load & Stress Testing (JMeter / k6)

Git Branch: `test/P8-04-performance` · Target Pull Request: test/P8-04-performance

Core Objective & Technical Scope:
Concurrency stress testing of financial ledger transfers and parking spot reservations under simulated peak campus traffic.

Priority Level	Medium	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`test/P8-04-performance`
Upstream Dependencies	P7-06		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author load testing scenarios simulating 500 concurrent booking attempts.
2. Execute load tests against containerized PostgreSQL and backend environment.
3. Analyze database lock contention and optimize connection pool parameters.
4. Document latency percentiles (p95, p99) under peak load.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `backend/src/test/**`
- `load scripts`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Zero balance drift or negative balances under maximum concurrency.
- Zero double-bookings permitted by database exclusion constraints.

VERIFICATION & TEST COVERAGE STRATEGY

- JMeter/k6 load test execution
- Database integrity post-load audit

TASK SPECIFICATION: P8-05 PHASE P8 — SECURITY HARDENING, COMPLIANCE & QUALITY ASSURANCE

Saudi PDPL Data Privacy & Audit Log Compliance Review

Git Branch: `chore/P8-05-privacy` · Target Pull Request: chore/P8-05-privacy

Core Objective & Technical Scope:
Verification of compliance with Saudi Personal Data Protection Law (PDPL), data minimization, and audit log scrubbing.

Priority Level	Medium	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`chore/P8-05-privacy`
Upstream Dependencies	P7-06		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Conduct complete audit of stored student personal identifiable information (PII).
2. Verify that passwords, tokens, and national IDs are masked in all application logs.
3. Document institutional data retention and deletion policies.
4. Author comprehensive docs/privacy.md compliance report.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `docs/privacy.md`
- `logging configurations`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- PDPL compliance checklist completed and verified.
- Zero unmasked PII present in production application logs.

VERIFICATION & TEST COVERAGE STRATEGY

- Log scrubbing verification test
- Compliance audit review

TASK SPECIFICATION: P9-01 PHASE P9 — PRODUCTION DEPLOYMENT, CI/CD & OPERATIONS

Production Orchestration & Secrets Management

Git Branch: `chore/P9-01-prod-config` · Target Pull Request: chore/P9-01-prod-config

Core Objective & Technical Scope:

Hardened production Docker Compose configuration, TLS termination, reverse proxy rules, and externalized secrets injection.

Priority Level	High	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`chore/P9-01-prod-config`
Upstream Dependencies	P8-05		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author hardened docker-compose.prod.yml with non-root security contexts.
2. Configure Nginx reverse proxy with TLS certificate termination and HTTP/2.
3. Externalize all production secrets via environment variables.
4. Document complete production deployment runbook.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `docker-compose.prod.yml`
- `docs/runbooks/deployment.md`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Zero secrets or sensitive defaults stored in git repository.
- Production stack starts cleanly with all services healthy.

VERIFICATION & TEST COVERAGE STRATEGY

- Production compose startup drill
- TLS certificate configuration test

TASK SPECIFICATION: P9-02 PHASE P9 — PRODUCTION DEPLOYMENT, CI/CD & OPERATIONS

Continuous Deployment (CD) Pipeline & Staging Delivery

Git Branch: `ci/P9-02-cd` · Target Pull Request: ci/P9-02-cd

Core Objective & Technical Scope:
Automated GitHub Actions deployment pipeline building verified production images and deploying to staging environment.

Priority Level	High	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`ci/P9-02-cd`
Upstream Dependencies	P9-01		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Configure GitHub Actions CD workflow triggered on main branch tags.
2. Build and tag multi-arch production Docker images.
3. Deploy to staging target environment via SSH/Webhook.
4. Execute automated smoke verification tests post-deployment.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `.github/workflows/cd.yml`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Automated deployment completes without manual intervention.
- Failing smoke test triggers automatic deployment rollback.

VERIFICATION & TEST COVERAGE STRATEGY

- CD pipeline execution drill
- Post-deploy smoke test

TASK SPECIFICATION: P9-03 PHASE P9 — PRODUCTION DEPLOYMENT, CI/CD & OPERATIONS

Automated Database Backup, Disaster Recovery & Health Monitoring

Git Branch: `chore/P9-03-backup-monitoring` · Target Pull Request: chore/P9-03-backup-monitoring

Core Objective & Technical Scope:
Automated PostgreSQL backup cron jobs, verified point-in-time recovery runbook, and Prometheus/Grafana metric telemetry.

Priority Level	High	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`chore/P9-03-backup-monitoring`
Upstream Dependencies	P9-01		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Author automated pg_dump backup script with encryption and compression.
2. Implement backup rotation policy with automated pruning.
3. Perform and document complete disaster recovery restoration drill.
4. Configure Prometheus actuator metric scraping and alert rules.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `scripts/backup*`
- `docs/runbooks/backup-restore.md`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- Database restoration drill proves clean recovery with zero data loss.
- Automated alerts trigger on container downtime or high resource utilization.

VERIFICATION & TEST COVERAGE STRATEGY

- Disaster recovery restoration drill
- Alert notification test

TASK SPECIFICATION: P9-04 PHASE P9 — PRODUCTION DEPLOYMENT, CI/CD & OPERATIONS

Operational Runbooks Completion & Milestone v1.0 Release

Git Branch: `chore/P9-04-release-1.0` · Target Pull Request: chore/P9-04-release-1.0

Core Objective & Technical Scope:
Finalization of operational procedures, maintenance runbooks, CHANGELOG publication, and official v1.0 semantic release tagging.

Priority Level	High	Complexity Rating	`M`
Execution Status	Backlog 📋	Target PR / Commit	`chore/P9-04-release-1.0`
Upstream Dependencies	P9-02, P9-03		
Architecture Blockers	None (Ready for execution)		

IMPLEMENTATION ROADMAP & ENGINEERING STEPS

1. Finalize incident response, maintenance, and upgrade runbooks.
2. Compile comprehensive CHANGELOG.md documenting all delivered capabilities.
3. Create official git release tag v1.0.0.
4. Conduct final production verification and formal handover.

TARGET FILE INVENTORY & EXPECTED ARTIFACTS

- `docs/runbooks/*`
- `CHANGELOG.md`

ACCEPTANCE CRITERIA & QUALITY CHECKLIST

- All operational runbooks validated by peer engineering review.
- Milestone v1.0 published and deployed successfully.

VERIFICATION & TEST COVERAGE STRATEGY

- Full system regression suite
- Production smoke test pass